

Análisis Predictivo.

Predicción de Variables Continuas..

M.C. Said Zamora.

Lineal.

Modelos lineales.

- La ecuación que describe una línea en un plano cartesiano.
- $y = mx + b$
- donde b es la ordenada al origen (el punto donde la línea interseca con la línea vertical en $x = 0$), y m es la pendiente (que describe la dirección e inclinación de la línea).
- En la regresión lineal, la ecuación que describe y como una función de x se escribe como:
- $y = b_0 + b_1x$
- donde b_0 (a veces β_0) es la ordenada al origen, y b_1 (a veces β_1) es la pendiente.
- Colectivamente, las b se conocen como los coeficientes beta.

Mínimos cuadrados.

- El método de mínimos cuadrados es un método de estimación de parámetros basado en minimizar la suma de los cuadrados de los residuos.
- Un residuo es la diferencia entre un valor observado y el valor ajustado proporcionado por un modelo.

$$m = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}$$

$$b = \frac{\sum y - m \sum x}{N}$$

Medición del error.

- El error es la diferencia entre el valor que nuestro modelo predice y el valor empírico real para cada observación
- Este es el término que se minimiza al encontrar la línea de mejor ajuste.
- Si el RSS es la suma de los residuos al cuadrado (o términos de error), la media de los residuos al cuadrado se conoce como el Error Cuadrático Medio (MSE, por sus siglas en inglés), y es una medida muy importante de la precisión de un modelo.

$$RSS = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$MSE = \frac{1}{n} \sum_{i=1}^n \varepsilon_i^2$$

Mínimos Cuadrados. (R)

- `modelo <- lm(data$waiting ~ data$eruptions, data = data)`
- `summary(modelo)`
- `abline(modelo)`

Heterocedasticidad.

- Dispersión desigual de los residuos o términos de error.
- Se refiere al caso donde hay un cambio sistemático en la dispersión de los residuos sobre el rango de valores medidos.
- Es un problema porque la regresión de mínimos cuadrados ordinarios (OLS) asume que los residuos provienen de una población que tiene homocedasticidad, lo que significa varianza constante.

Prueba de Breusch-Pagan.

- Se utiliza para determinar si la heterocedasticidad está presente o no en un modelo de regresión.
- La prueba utiliza las siguientes hipótesis nulas y alternativas:
- Hipótesis Nula (H_0): Está presente la homocedasticidad (los residuos se distribuyen con varianza constante).
- Hipótesis Alternativa (H_A): Está presente la heterocedasticidad (los residuos no se distribuyen con varianza constante).
- Si el valor p que corresponde a este estadístico de prueba Chi-Cuadrado con p (el número de predictores) grados de libertad es menor que algún nivel de significancia (es decir, $\alpha = 0.05$), entonces rechazamos la hipótesis nula y concluimos que la heterocedasticidad está presente.

Prueba de Breusch-Pagan. (R)

- `bptest(modelo)`

Regresión lineal simple con un predictor binario.

- Normalmente, en la ecuación de regresión lineal simple
- $Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$
- X_i representa el valor real de la variable predictora. En el caso de una regresión lineal simple con una variable predictora binaria (), X_i será una variable ficticia.
- Cuando el predictor es dado un valor, X_i será 0, y cuando el predictor es el valor opuesto, X_i será 1.

RLSPB. (R)

- `model <- lm(mpg ~ am, data=mtcars)`
- `summary(model)`
- `t.test(mpg ~ am, data=mtcars,`
- `var.equal=TRUE)`

RLSPB. (R)

- `mtcars$automatic <- ifelse(mtcars`
- `+                      $am==0, "yes", "no")`
- `model <- lm(mpg ~ factor(automatic),`
- `+              data=mtcars)`
- `model`

Predicción de Variables Continuas.

Regresión Múltiple.

Regresión Múltiple.

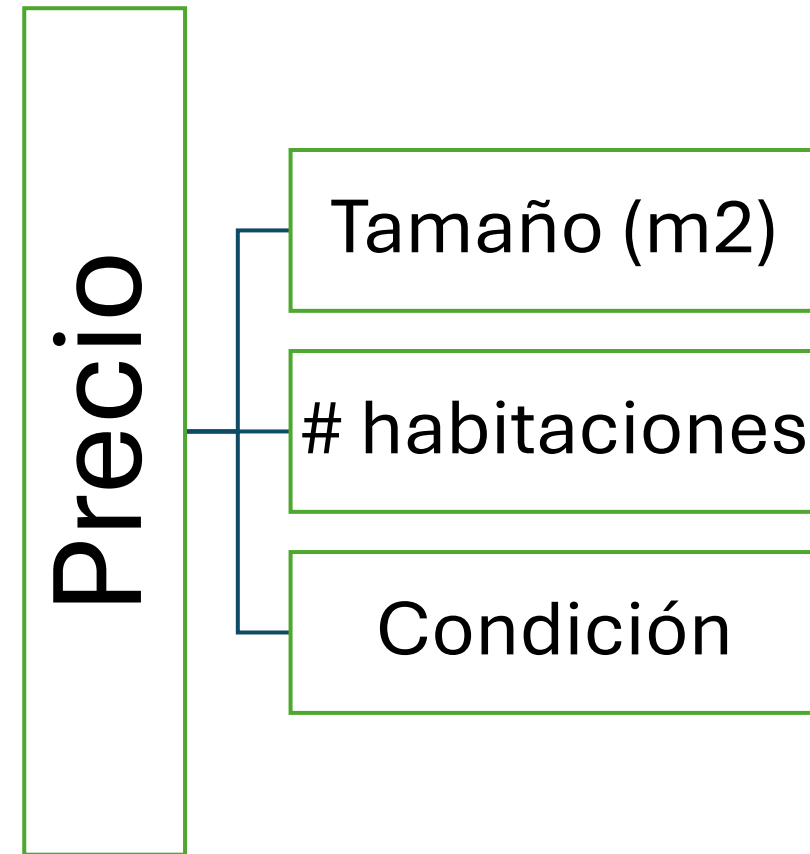
- La regresión múltiple es una extensión de la regresión lineal simple donde se utilizan múltiples variables predictoras para explicar la variación en la variable dependiente.

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \varepsilon$$

Regresión Múltiple.

- Predecir el precio de una vivienda (variable dependiente) en función del tamaño (en metros cuadrados), número de habitaciones y su condición (variables predictoras).

$$\hat{Y} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$$



Kitchen Sink.

- Regresión en la que se incluyen todas las variables predictoras disponibles en el modelo.
- Se utiliza para exploración inicial o cuando hay una fuerte justificación teórica para incluir todas las variables.
- Puede llevar a problemas como:
- Sobreajuste: El modelo puede ajustarse demasiado bien a los datos de entrenamiento, pero tener un rendimiento deficiente en datos nuevos.
- Multicolinealidad: Las variables predictoras pueden estar altamente correlacionadas entre sí, lo que dificulta la interpretación de sus efectos individuales.

Entrenamiento de un modelo de regresión lineal múltiple.

- Preparación de datos.
- Selección de características.
- Entrenamiento del modelo.
- Evaluación del rendimiento.
- Refinamiento del modelo.

Preparación de datos.

- Recopilación de datos: Se reúnen datos relevantes para el problema. Se asegura de tener suficientes observaciones y que las variables sean relevantes para predecir la variable objetivo.
- Limpieza de datos: Se manejan los valores faltantes, los valores atípicos y las inconsistencias en los datos.
- Transformación de datos:
 - Escalado: Se estandarizan las variables independientes para que tengan una escala similar. Esto puede mejorar la convergencia del algoritmo de entrenamiento.
 - Codificación: Se convierten las variables categóricas en representaciones numéricas.
 - Ingeniería de características: Se crean nuevas características que puedan ser informativas para el modelo.

Selección de características.

- Análisis de correlación: Se examinan las correlaciones entre las variables independientes y la variable objetivo. Las variables con alta correlación pueden ser más informativas.
- Selección manual: Basándose en el conocimiento del dominio, se seleccionan las características que son más relevantes.
- Selección automática: Utiliza métodos como la regresión stepwise, Lasso o Ridge para seleccionar automáticamente las características más importantes.

Regresión Stepwise.

- Técnica automatizada para seleccionar variables predictoras en un modelo de regresión.
- Forward Selection: Comienza con un modelo vacío y agrega variables una por una, eligiendo en cada paso la que más mejora el modelo según un criterio.
- Backward Elimination: Comienza con un modelo que incluye todas las variables y elimina una por una, eligiendo en cada paso la que menos contribuye al modelo.
- Bidirectional Elimination (Stepwise): Combina ambos, agregando y eliminando variables en cada paso.

Criterio de Información de Akaike (AIC).

- Medida de la calidad relativa de un modelo estadístico para un conjunto de datos dado.
- Ayuda a comparar diferentes modelos y elegir el que mejor se ajuste a los datos, teniendo en cuenta tanto la bondad de ajuste como la complejidad del modelo.
 - $AIC = 2k - 2\ln(L)$
- k es el número de parámetros estimados en el modelo.
- L es el valor máximo de la función de verosimilitud del modelo (una medida de qué tan bien el modelo se ajusta a los datos)

Criterio de Información de Akaike (AIC).

- Un valor más bajo de AIC indica un mejor modelo.
- El AIC penaliza los modelos con más parámetros para evitar el sobreajuste.
- El AIC no te dice si un modelo es "bueno" en términos absolutos, sino que te ayuda a comparar modelos relativos entre sí.

Regresión Stepwise. (R)

- `library(MASSExtra)`
- `data(Boston)`
- `full_model <- lm(medv ~ ., data = Boston)`
- `# Regresión stepwise bidireccional (AIC como criterio)`
- `step_model <- stepAIC(full_model, direction = "both", trace = FALSE)`
- `summary(step_model)`

Regresión Stepwise. (R)

- `library(MASSExtra)`
- `full_model <- lm(precio ~ ., data = data)`
- `# Regresión stepwise bidireccional (AIC como criterio)`
- `step_model <- stepAIC(full_model, direction = "both", trace = FALSE)`
- `summary(step_model)`

Entrenamiento del modelo.

- División de datos: Se dividen los datos en un conjunto de entrenamiento y un conjunto de prueba. El conjunto de entrenamiento se utiliza para ajustar los parámetros del modelo y el conjunto de prueba para evaluar su rendimiento.
- Elección del algoritmo: La regresión lineal múltiple es un algoritmo comúnmente utilizado. También puedes considerar variantes como la regresión Ridge o Lasso si tienes muchas características o sospechas de multicolinealidad.
- Ajuste del modelo: Se utiliza el algoritmo elegido para encontrar los coeficientes que mejor se ajustan a los datos de entrenamiento. Esto implica minimizar una función de pérdida (como el error cuadrático medio).

Parámetro: tamaño muestra para pruebas.

- Si su valor es 0.2 indica que el 20% del conjunto de datos se debe separar y utilizar como conjunto de prueba. El restante 80% se utilizará para entrenar el modelo.
- Seleccionar 0.2 significa que se busca un equilibrio entre tener suficientes datos para entrenar el modelo de manera efectiva y al mismo tiempo tener suficientes datos para probar y evaluar el rendimiento del modelo.
- Es importante tener un conjunto de prueba representativo, ya que proporciona una estimación imparcial del rendimiento del modelo en datos no vistos.
- Si el conjunto de datos es muy grande, se puede utilizar un valor más pequeño, como 0.1 o incluso menos.
- Si el conjunto de datos es pequeño, es posible necesitar un valor más grande para asegurarte de que el conjunto de prueba sea lo suficientemente grande para ser representativo.

Evaluación del rendimiento

- Métricas de evaluación: Se utilizan métricas como el error cuadrático medio (MSE), el error absoluto medio (MAE) o el R^2 para evaluar qué tan bien el modelo predice la variable objetivo en el conjunto de prueba.
- Validación cruzada: Se aplica la validación cruzada (como la validación cruzada k-fold) para obtener una estimación más robusta del rendimiento del modelo y evitar el sobreajuste.

Regresión Múltiple. (R)

- `data <- read.csv(file.choose())`
- `model <- lm(precio ~ tcons + recamaras + condicion, data = data)`
- `> summary(model)`
- `# Predicciones con nuevos datos`
- `new_data <- data.frame(tcons = 130, recamaras = 3, condicion = 4)`
- `predict(model, new_data)`

Regresión Lasso.

- La regresión Lasso (Least Absolute Shrinkage and Selection Operator) es una técnica de regresión lineal que incorpora regularización.
- Esto significa que añade una penalización al tamaño de los coeficientes del modelo, lo que ayuda a prevenir el sobreajuste y puede llevar a que algunos coeficientes se vuelvan exactamente cero, actuando como una forma de selección de características.

Regresión Lasso. (R)

- `library(glmnet)`
- `datos <- read.csv(file.choose())`
- `x <- model.matrix(precio~., data = datos)[,-1]`
- `y <- datos$precio # Variable respuesta`
- `modelo_lasso <- glmnet(x, y, alpha = 1) # alpha = 1 para Lasso`
- `cv_lasso <- cv.glmnet(x, y, alpha = 1)`
- `mejor_lambda <- cv_lasso$lambda.min`
- `coeficientes <- coef(modelo_lasso, s = mejor_lambda)`
- `print(coeficientes)`
- `coef(modelo_lasso, s = mejor_lambda)`
`prediccionesl <- predict(modelo_lasso, newx = x, s = mejor_lambda)`
`mse <- mean((datos$precio - prediccionesl)^2)`
- `print(paste("MSE:", mse))`

Regresión Ridge.

- Añade una penalización al tamaño de los coeficientes en la función de costo de la regresión lineal.
- Esta penalización, controlada por un parámetro de regularización (λ o α), "encoge" los coeficientes hacia cero, reduciendo su varianza y mejorando la generalización del modelo.

Regresión Ridge. (R)

- `X_matrix <- model.matrix(precio~., data = datos)[,-1]`
- `alpha <- 1.0 # Parámetro de regularización`
- `ridge_model <- glmnet(X_matrix, datos$precio, alpha = alpha)`
- `cv_fit <- cv.glmnet(X_matrix, datos$precio, alpha = alpha)`
- `best_lambda <- cv_fit$lambda.min`
- `best_lambda`
- `coef(ridge_model, s = best_lambda)`
- `predicciones <- predict(ridge_model, newx = X_matrix, s = best_lambda)`
- `mse <- mean((datos$precio - predicciones)^2)`
- `print(paste("MSE:", mse))`

Regresión con Predictor No Binario.

- A diferencia de los predictores binarios (que solo toman dos valores, como "sí" o "no"), los predictores no binarios pueden tener múltiples categorías o ser variables continuas.

Regresión con Predictor No Binario. (R)

- `datos <- read.csv(file.choose())`
- `datos$condicion <- as.factor(datos$condicion)`
- `modelo <- lm(precio ~ tcons + condicion + tterreno + numban +
recamaras, data = datos)`
- `summary(modelo)`

Regresión con Predictor No Binario. (R)

- `datos$recamaras <- as.factor(datos$recamaras)`
- `modelo <- lm(precio ~ tcons + condicion + tterreno + numban + recamaras, data = datos)`
- `summary(modelo)`

Regresión logística.

Regresión logística.

- Modelo estadístico utilizado para predecir la probabilidad de que un evento ocurra en función de los valores de un conjunto de variables independientes.
- A diferencia de la regresión lineal, que predice valores continuos, la regresión logística predice la probabilidad de pertenecer a una categoría específica.
-
- La regresión logística utiliza la función sigmoidea para transformar los valores lineales en probabilidades.
- El resultado de la función logística se interpreta como la probabilidad de que la observación pertenezca a la categoría de interés.

Tipos de regresión logística,

- Regresión logística binaria: Predice la probabilidad de que un evento ocurra (1) o no ocurra (0).
- Regresión logística multinomial: Predice la probabilidad de pertenecer a una de varias categorías posibles.

Factor de Inflación de la Varianza (VIF).

- Mide cuánto aumenta la varianza de un coeficiente de regresión estimado debido a la colinealidad (correlación) con otros predictores en el modelo.
- Un VIF alto indica que la multicolinealidad podría estar afectando la estimación de ese coeficiente, haciéndolo menos confiable.
- Un VIF de 1 indica ausencia de colinealidad. Valores entre 1 y 5 sugieren una colinealidad moderada, que podría no ser problemática. Valores superiores a 5 o 10 suelen considerarse indicativos de una colinealidad preocupante.

Regresión logística binaria (R).

- `library(glmnet)`
- `median_precio <- median(df$precio)`
- `df$precio_alto <- as.integer(df$precio > median_precio)`
- `model <- glm(precio_alto ~ recamaras + baños + m2_construcción + m2_terreno, data = df, family = binomial)`
- `summary(model)`
- `library(car)`
- `Anova(model)`
- `vif(model)` # multicollinearity (variance inflation factor)

Regresión logística multinomial (R).

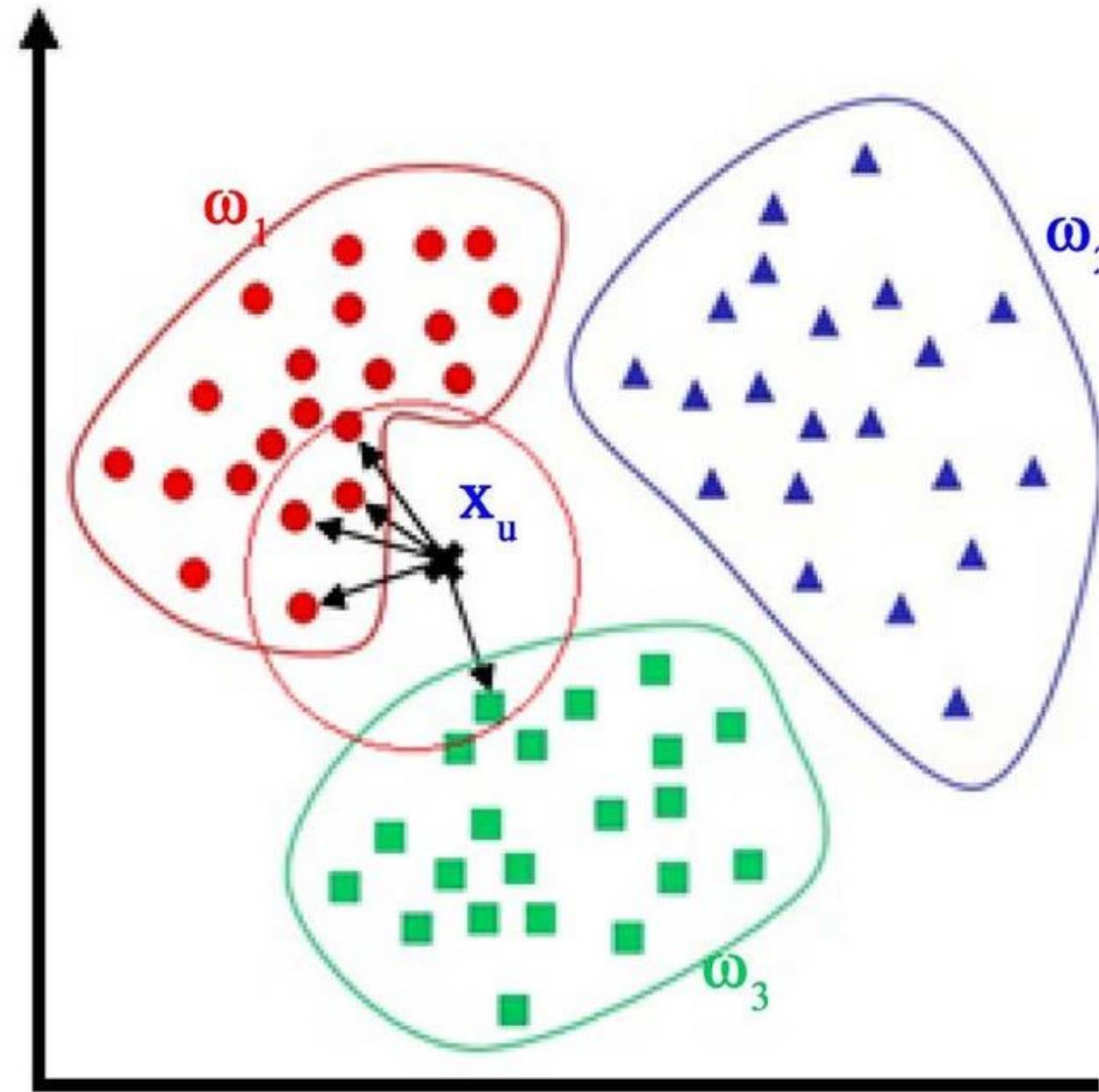
- `library(nnet)`
- `percentile_33 <- quantile(df$precio, 0.33)`
- `percentile_66 <- quantile(df$precio, 0.66)`
- `df$categoria_precio <- cut(df$precio, breaks = c(-Inf, percentile_33, percentile_66, Inf), labels = c("bajo", "medio", "alto"), right = FALSE)`
- `model_multinom <- multinom(categoria_precio ~ recamaras + baños + m2_construcción + m2_terreno, data = df)`
- `summary(model_multinom)`
- `coefficients <- coef(model_multinom)`
- `intercepts <- model_multinom$wts[1:nlevels(df$categoria_precio)]`
- `print(coefficients)`
- `new_data <- data.frame(recamaras = 3, baños = 2, m2_construcción = 120, m2_terreno = 200)`
- `predicted_probabilities <- predict(model_multinom, newdata = new_data, type = "probs")`
- `print(predicted_probabilities)`

Predicción de Variables Categorías.

K Vecinos Más Cercanos
(KNN).

K Vecinos Más Cercanos (KNN).

- Algoritmo de clasificación y regresión que predice la clase o el valor de un nuevo punto de datos basándose en los "k" puntos de datos más cercanos en el espacio de características.



K = desconocido.

Suma de Cuadrados Dentro de los Grupos (WCSS).

- Cuantifica qué tan agrupados están los puntos de datos dentro de sus grupos.
- Calcula la suma de las distancias al cuadrado entre cada punto de datos en un grupo y su centroide (el centro del grupo).
- Un WCSS más bajo indica que los puntos de datos están más cerca del centroide de su grupo, lo que sugiere un grupo bien definido.

$$WCSS = \sum \|x - \mu\|^2$$

Método del codo.

- Técnica gráfica utilizada en el análisis de conglomerados (clustering) para determinar el número óptimo de grupos (clusters) en un conjunto de datos.
- Se crea un gráfico donde el eje x representa el número de grupos (k) y el eje y representa la medida de distancia.
- A medida que aumentas el número de grupos, el WCSS disminuye, en el punto donde la disminución se vuelve menos pronunciada, formando una especie de "codo" en el gráfico, ese punto de codo se considera el número óptimo de grupos, ya que agregar más grupos no reduce significativamente la variabilidad dentro de los grupos.

KNN (R).

- `library(class)`
- `library(dplyr)`
- `df <- read.csv(file.choose())`
- `median_price <- median(df$precio)`
- `df$categoria_precio <- ifelse(df$precio >= median_price, 'alto', 'bajo')`
- `df_selected <- df %>% select(recamaras, baños, m2_construcción, m2_terreno, pisos, condición, categoria_precio)`
- `set.seed(42)`
- `train_index <- sample(1:nrow(df_selected), 0.75 * nrow(df_selected))`
- `train_data <- df_selected[train_index, ]`
- `test_data <- df_selected[-train_index, ]`
- `knn_model <- knn(train = train_data %>% select(-categoria_precio),`
- `test = test_data %>% select(-categoria_precio),`
- `cl = train_data$categoria_precio, k = 3)`
- `predictions <- knn_model`
- `head(predictions, 5)`

Matriz de confusión.

- Herramienta para evaluar el rendimiento de un modelo de clasificación.
- tp (verdaderos positivos): el número de instancias que se clasifican correctamente como positivas.
- fn (falsos negativos): el número de instancias que se predicen como negativas, pero deberían haberse clasificado como positivas.
- fp (falsos positivos): el número de instancias que se predicen como positivas, pero deberían haberse clasificado como negativas.
- tn (verdaderos negativos): el número de instancias que se clasifican correctamente como negativas.

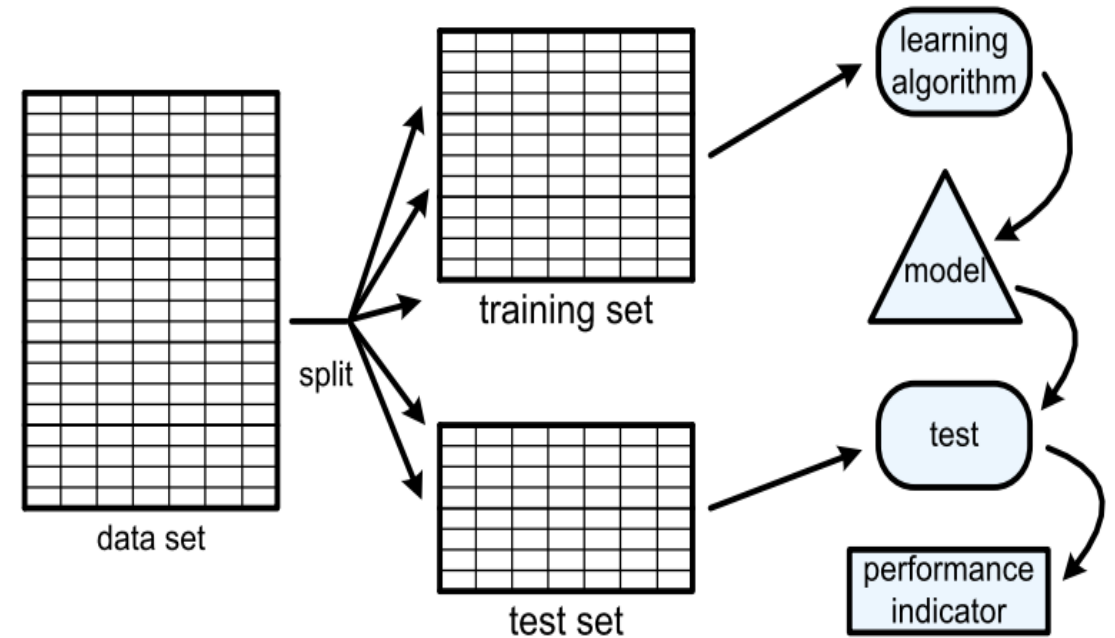
		<i>predicted class</i>		
		+	-	
<i>actual class</i>	+	<i>tp</i>	<i>fn</i>	<i>p</i>
	-	<i>fp</i>	<i>tn</i>	<i>n</i>
		<i>p'</i>	<i>n'</i>	<i>N</i>

Matriz de confusión (R).

- `library(caret)`
- `test_data$Prediccion <- predictions`
- `cm <- confusionMatrix(table(test_data$categoria_precio, test_data$Prediccion))`
- `print("Matriz de Confusión:")`
- `print(cm$table)`
- `print("\nOtras Métricas:")`
- `print(cm$byClass)`
- `accuracy <- sum(diag(cm$table)) / sum(cm$table)`
- `print(paste("\nAccuracy:", accuracy))`

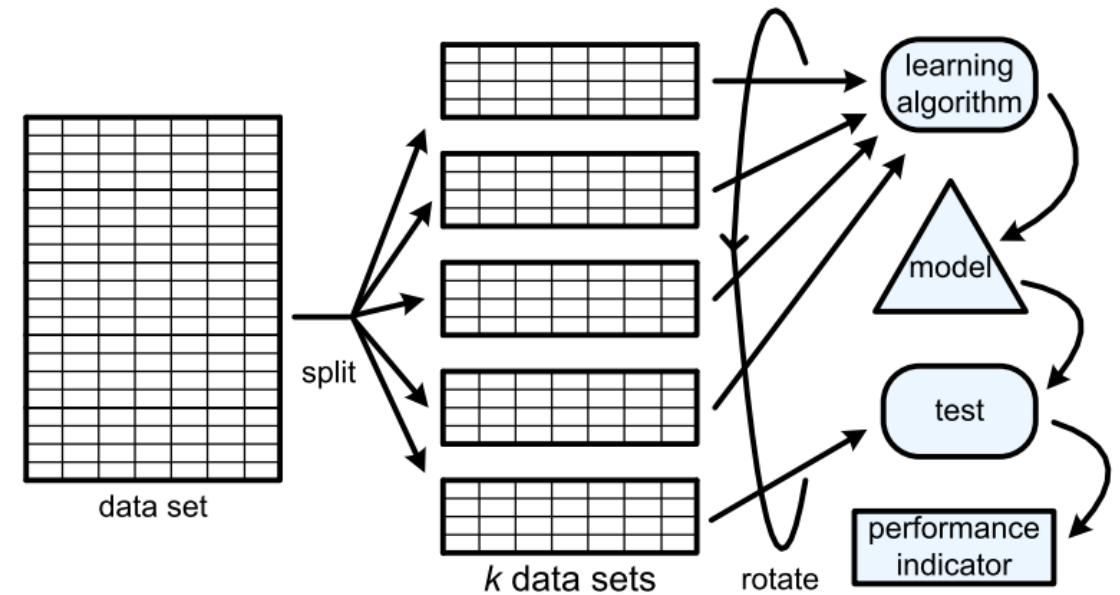
Validación cruzada.

- Técnica que se utiliza para evaluar qué tan bien se espera que un modelo estadístico generalice a un conjunto de datos independiente.
- Ayuda a estimar el rendimiento del modelo en datos nuevos que no se utilizaron para entrenarlo.
- División de datos: El conjunto de datos original se divide en varios subconjuntos (o pliegues).
- Entrenamiento y evaluación iterativa: En cada iteración, se selecciona un pliegue como conjunto de prueba y los demás pliegues se combinan para formar el conjunto de entrenamiento. El modelo se entrena en el conjunto de entrenamiento y luego se evalúa su rendimiento en el conjunto de prueba.
- Cálculo del rendimiento promedio: Se repite el proceso para cada pliegue, utilizándolo como conjunto de prueba una vez y se calcula el promedio de los rendimientos obtenidos en cada iteración.



Tipos de Validación cruzada.

- Validación cruzada de k pliegues (k-fold): el conjunto de datos se divide en k pliegues.
- Validación cruzada leave-one-out (LOO): Cada muestra se utiliza como conjunto de prueba una vez, y el resto de las muestras forman el conjunto de entrenamiento.
- Validación cruzada repetida: Se repite la validación cruzada de k pliegues varias veces con diferentes particiones aleatorias de los datos.



Determinación k (R).

- `k_values <- seq(1, 15, by = 2)`
- `cv_results <- train(categoria_precio ~ ., data = train_data, method = "knn", tuneGrid = data.frame(k = k_values), trControl = trainControl(method = "cv", number = 10))`
- `best_k <- cv_results$bestTune$k`
- `print(paste("Mejor valor de k:", best_k))`
- `best_accuracy <- max(cv_results$results$Accuracy)`
- `print(paste("Precisión del mejor k:", best_accuracy))`
- `plot(cv_results)`

Bosques aleatorios.

- Se basa en la idea de construir una multitud de árboles de decisión y combinar sus resultados para obtener una predicción más precisa y robusta.
- Muestreo: Se toman múltiples muestras aleatorias del conjunto de datos original, con reemplazo (lo que significa que un mismo dato puede aparecer varias veces en una muestra).
- Construcción de árboles: Para cada muestra, se construye un árbol de decisión. En cada nodo del árbol, se selecciona aleatoriamente un subconjunto de características para determinar la mejor división.
- Predicción: Para hacer una predicción, se pasa la nueva instancia por cada uno de los árboles y se obtiene una predicción de cada uno. En el caso de clasificación, la predicción final es la clase más votada por los árboles. En el caso de regresión, es el promedio de las predicciones de los árboles.

Bosques aleatorios (R).

- `library(randomForest)`
- `X <- df[, !(colnames(df) %in% 'precio')]`
- `y <- df$precio`
- `set.seed(42) # For reproducibility`
- `train_indices <- sample(nrow(df), 0.8 * nrow(df))`
- `X_train <- X[train_indices, ]`
- `X_test <- X[-train_indices, ]`
- `y_train <- y[train_indices]`
- `y_test <- y[-train_indices]`
- `model <- randomForest(x = X_train, y = y_train, ntree = 100, importance = TRUE) # Start with 100 trees`
- `y_pred <- predict(model, newdata = X_test)`
- `rmse <- sqrt(mean((y_test - y_pred)^2))`
- `mae <- mean(abs(y_test - y_pred))`
- `r_squared <- 1 - sum((y_test - y_pred)^2) / sum((y_test - mean(y_test))^2)`
- `cat("\nPerformance Measures:\n")`
- `cat("RMSE:", rmse, "\n")`
- `cat("MAE:", mae, "\n")`
- `cat("R-squared:", r_squared, "\n\n")`
- `varImpPlot(model)`